

Correctness and Testing Workshop

ICCS Summer School 2026

Dominic Orchard and Nick Huber

Thanks also to Isaac Akanho

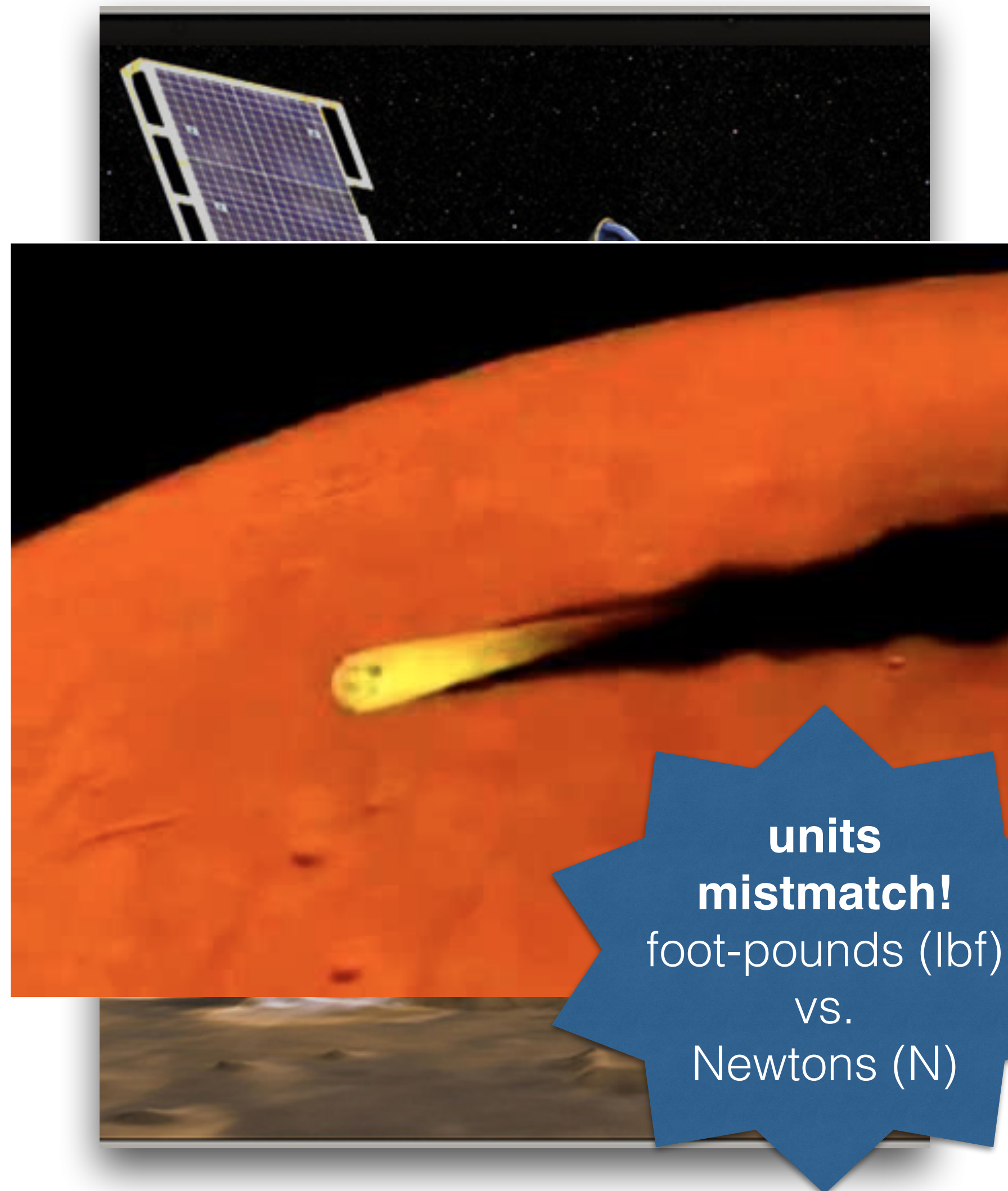


UNIVERSITY OF
CAMBRIDGE



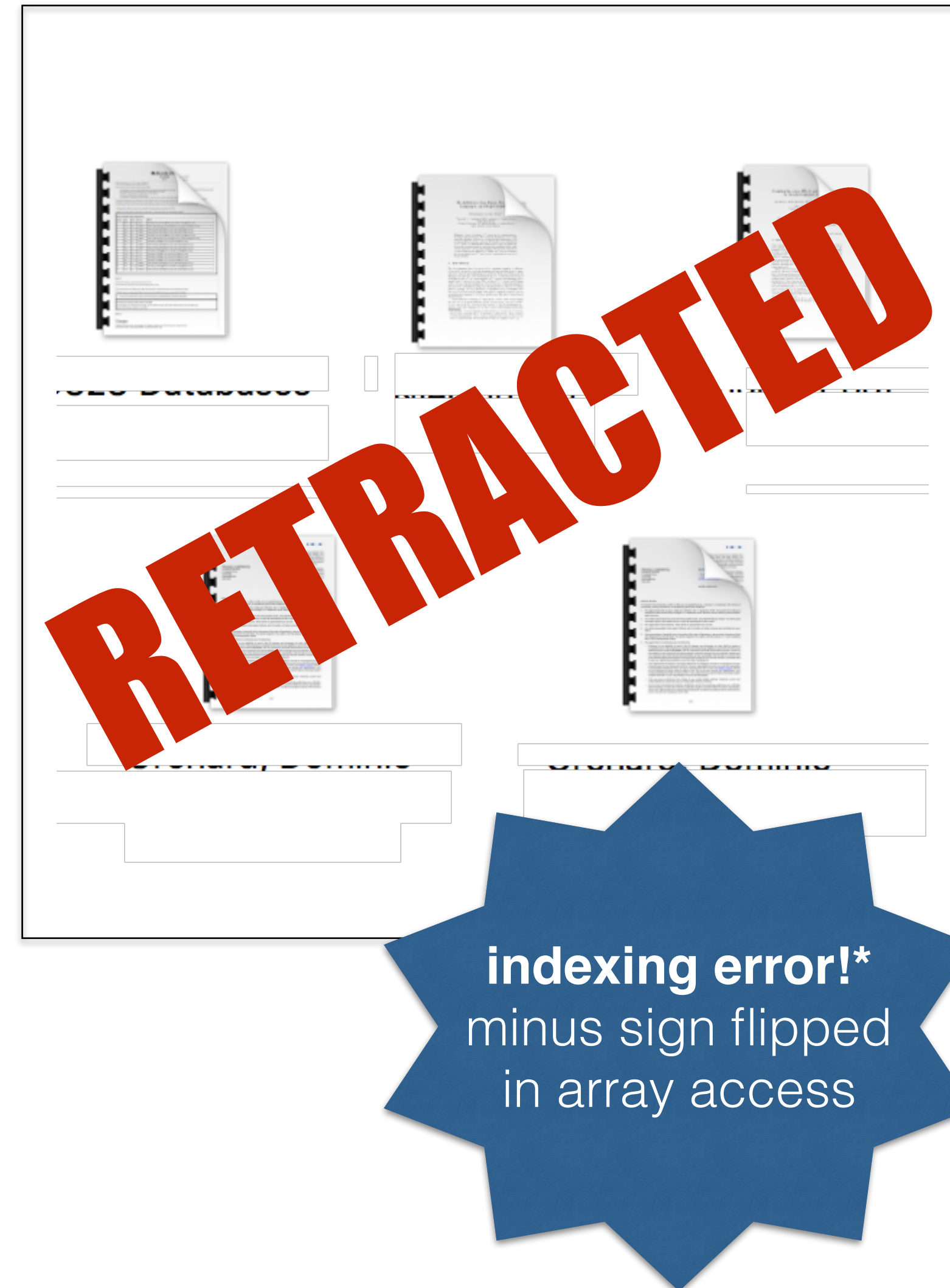
Institute of
Computing for
Climate Science

Schmidt Sciences



**units
mismatch!**
foot-pounds (lbf)
vs.
Newtons (N)

NASA/JPL/Corby Waste



indexing error!*
minus sign flipped
in array access

average bug-rate in industry software is 15-50 errors per 1000 lines**

* Z. Merali, Computational science: Error, why scientific programming does not compute, 2010

** S. McConnell, *Code complete*, O'Reilly Media, Inc., 2004.

Validation

Did we implement the right equations?

VS

Verification

Did we implement the equations right?

Challenge

Telling these two apart when results are not as expected

Terminology: what does “*verified*” mean?

Verification wrt. a specification

i.e. `check(implementation, specification)`

*The value of a specification is what we make of it;
it depends on our goals, values, and **which risks we wish
to mitigate against***

Validation as a proxy for verification

- Can we get a stable run (over decades)?
- Is it plausible from physics perspective?
- Do hindcasts reproduce observational record?

If the science is relatively settled, then may point to bugs not invalidity

Problem: error localisation is poor!

How *much* verification?

- Lots of verification techniques out there:
 - Testing
 - Type systems
 - Deductive verification
 - Static analysis
 - Interactive theorem provers
 - Modelling and model checking
- How much to use?

Our focus: Testing

- Implementation language = specification language
- Test subset of inputs / program behaviours
 - Does not show “absence of bugs” (Dijkstra); may fail to expose unconceived-of bugs
- Key principles:
 - automate testing so that it is easy to deploy
 - speed up development time by reducing bug hunting time
 - increase confidence in code
 - enable safe(r) refactoring (*want semantics preservation*)

Plan

- We will use `python` and `pytest`

- Session 1

- Part 0 - Smoke tests
- Part 1 - Unit tests
- Exercises

- Session 2

- Part 2 - Integration tests
- Part 3 - Property-based tests
- Exercises

- Live demos throughout + exercises based on a 0D Energy Balance model
<https://github.com/Cambridge-ICCS/testing-workshop>

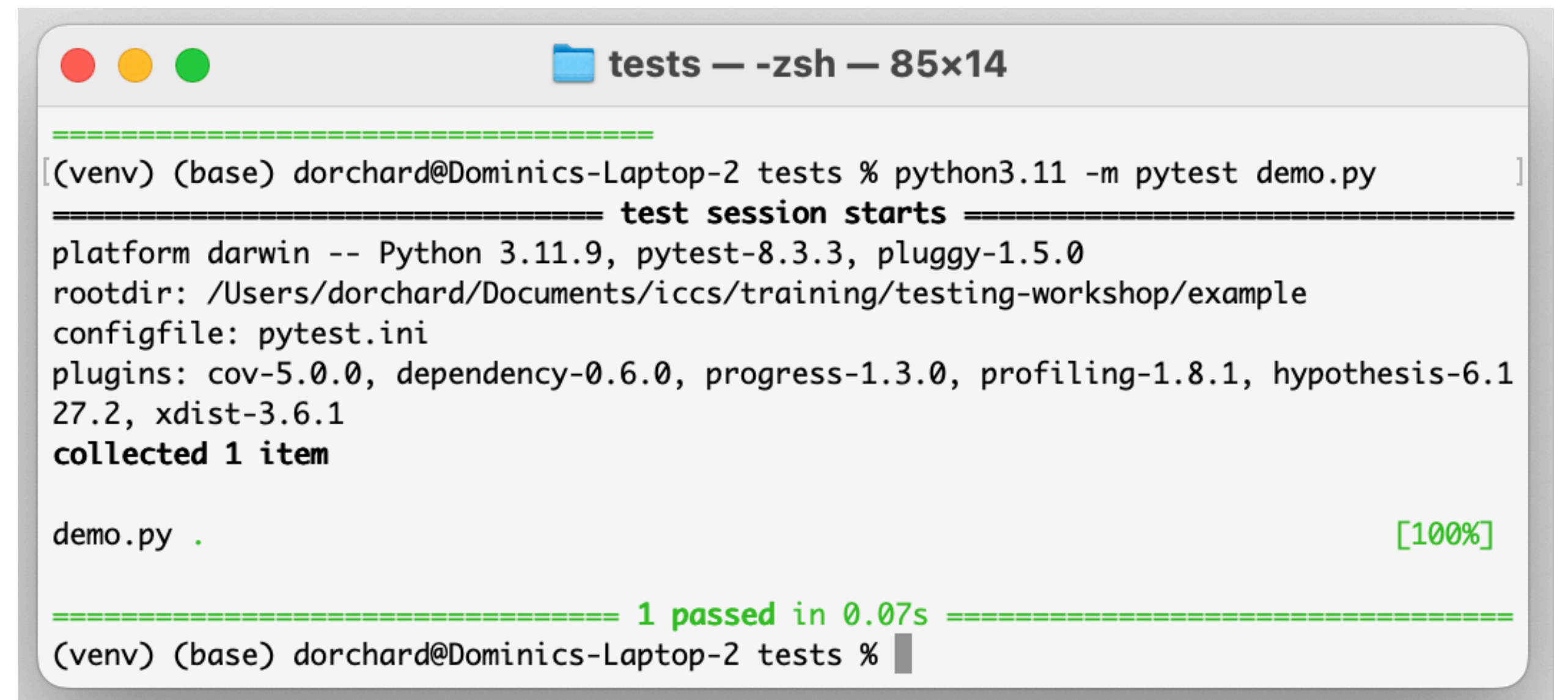


pytest basics

- Put tests as .py files in a `tests/` directory
- Each test group is a function with prefix `test_`
- Use `assert` keyword with a boolean expression

tests/demo.py

```
def func(x):  
    return x + 1  
  
def test_answer():  
    assert func(3) == 4
```

A terminal window titled "tests — -zsh — 85x14" showing the execution of a pytest command. The command is "python3.11 -m pytest demo.py". The output shows the test session starting, platform details (darwin), Python version (3.11.9), pytest version (8.3.3), and various plugins. It then shows "collected 1 item" and "demo.py ." with a green dot indicating success. At the bottom, it says "1 passed in 0.07s" with a green bar. The prompt is "(venv) (base) dorchard@Dominics-Laptop-2 tests %".

```
tests — -zsh — 85x14  
=====
```

```
[(venv) (base) dorchard@Dominics-Laptop-2 tests % python3.11 -m pytest demo.py ]  
===== test session starts =====  
platform darwin -- Python 3.11.9, pytest-8.3.3, pluggy-1.5.0  
rootdir: /Users/dorchard/Documents/iccs/training/testing-workshop/example  
configfile: pytest.ini  
plugins: cov-5.0.0, dependency-0.6.0, progress-1.3.0, profiling-1.8.1, hypothesis-6.1  
27.2, xdist-3.6.1  
collected 1 item  
  
demo.py . [100%]  
  
===== 1 passed in 0.07s =====  
(venv) (base) dorchard@Dominics-Laptop-2 tests %
```

assert statement anatomy

```
assert boolean-expr
```

```
assert boolean-expr, string-expr
```

Meaning: if `boolean-expr` **false**, show `string-expr` in the output

Part 0 - Smoke tests

What is a smoke test?

🔥 *Did it catch fire?*

- Was the program able to “start” (initialise) without throwing an exception?
 - *Check that module can be loaded*

```
def test_initialization():  
    from energy_balance import __init__ as package_init  
    assert package_init is not None, "Package initialization failed"
```

- *Might include loading configuration files or data sets*

```
def test_config_loading():  
    # Check that init does not fail  
    energy_balance.init()  
    # and that it initializes a global variable  
    assert energy_balance.solar_constant is not None, "Solar constant should be initialized"
```

Part I - Unit tests

Unit test basics (1)

- Applied to smaller “units” of code...
- Run code, then compare **actual** results against **expected** results
For example, conceptually....

square : float $\rightarrow$ float

square(2) $\equiv$ 4

Unit test basics (2)

- Try various values *systematically*:
 - with simple / base cases

square(0) $\equiv$ 0

square(1) $\equiv$ 1

- with more complex cases

square(4) $\equiv$ 16

square(-3) $\equiv$ 9

- with corner cases (*requires more creative thought sometimes*)

square(NaN)???

Unit test demo

tests/test_helpers.py

```
from helpers import square
from math import nan, isnan

def test_square():
    assert square(0) == 0
    assert square(1) == 1
    assert square(4) == 16
    assert square(-1) == 1
    assert isnan(square(nan))
```

(NaN $\neq$ NaN use `math.isnan` instead)

Floating point considerations...

- $\text{NaN} \neq \text{NaN}$ use `math.isnan` instead
- Floating-point rounding error can be a pain,
 - `pytest.approx`
 - e.g., `pytest.approx(0.614618, abs=1e-4)`
 - `rel` - Relative tolerance
 - `abs` - Absolute tolerance
 - OR `np.isclose` / `np.allclose`

Relative vs. absolute error

- Absolute error: $\epsilon_{\text{abs}} = |x - x'|$ (**typically** x = old/precise and x' = new/approx)
- Relative error: $\epsilon_{\text{rel}} = \frac{\epsilon_{\text{abs}}}{x}$ (contextualised/scaled error)

x	x'	abs	rel
1000000	1000000.1	0.1	1E-08
1	1.1	0.1	0.1
100	200	100	1
1	2	1	1



*Relative better when the scale
(across measurements) varies*

*Absolute can be useful if known
acceptable precision*

Test *parameterization*

Reduce repetition by defining a single test over different inputs

tests/test_helpers.py

```
from helpers import square
from math import nan, isnan

def test_square():
    assert square(0) == 0
    assert square(1) == 1
    assert square(4) == 16
    assert square(-1) == 1
    assert isnan(square(nan))
```



tests/test_helpers_param.py

```
from helpers import square
from math import nan, isnan
import pytest

def test_square_nan():
    assert isnan(square(nan))

@pytest.mark.parametrize("value, expected", [
    (0, 0),
    (1, 1),
    (4, 16),
    (-1, 1),
])
def test_square_parametrized(value, expected):
    assert square(value) == expected
```

Test fixtures

Provide context or content for tests

tests/conftest.py

```
import energy_balance
import pytest

@pytest.fixture
def setup():
    energy_balance.init()
```

Define a fixture

tests/test_energy_balance.py

```
import energy_balance

def test_energy_in_albedo_max(setup):
    assert energy_balance.energy_in(1) == 0
```

Use it in tests that need it

Negative tests

“We expect this to fail with an exception”

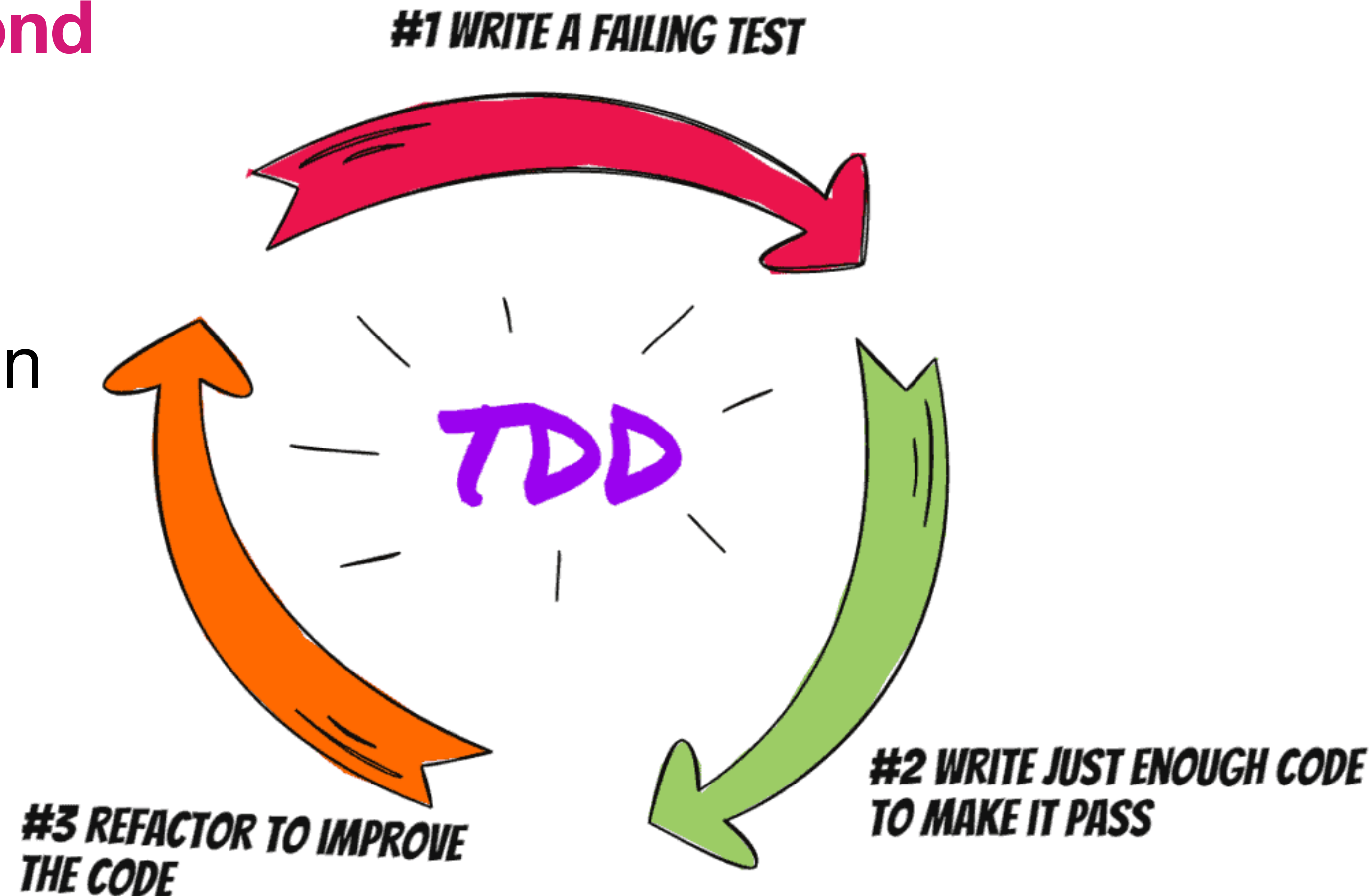
```
def test_energy_in_invalid(setup):  
    albedo = -0.1  
    with pytest.raises(ValueError):  
        energy_balance.energy_in(albedo)
```


Unit test basics (3)

- **General principle:** try to cover a representative sample of input space.
- For data structures:
 - Empty data structure
 - Small data structures
 - All constructor shapes (i.e., all small sizes of binary tree)
 - Very large?

Test-Driven Development (TDD)

- **Specify first, implement second**
- TDD cycle is quite fine grained
 - #1 could be repeated first
- Encourage more modular design
- ‘Correctness first’ mindset



Coverage

What % of *control-flow paths* have a test?

```
pytest test-path --cov code-path
```

e.g. `pytest tests --cov .`

To find out which control-flow paths
(per line) are not covered:

```
pytest test-path --cov code-path --cov-report term-missing
```

```
pytest test-path --cov code-path --cov-report html
```

```
----- coverage: platform darwin, python 3.11.9-final-0 -----
Name                               Stmts  Miss  Cover
-----
__init__.py                         0      0   100%
src/__init__.py                     1      1     0%
src/doctest_demo.py                 5      5     0%
src/energy_balance.py               65     10    85%
src/helpers.py                      5      0   100%
src/observation.py                  12      1    92%
tests/conftest.py                   5      0   100%
tests/smoke_test.py                 21      2    90%
tests/test_energy_balance.py         19      0   100%
tests/test_helpers.py               10      0   100%
tests/test_helpers_param.py         14      0   100%
tests/test_observation.py           28      0   100%
tests/test_observation_done.py       29      0   100%
-----
TOTAL                               214     19    91%

===== 26 passed, 2 warnings in 1.50s =====
(venv) (base) dorchard@edud455 example %
```


Exercises

Part II - Integration tests

(thanks to Isaac Akanho)

What is Integration Testing?

- Tests two or more components (units), modules or (sub)systems working together
- Focuses on the connections rather than isolated logic

What is a Component?

- A self-contained slice of logic
- May embed smaller components inside it
- An example of a minimal (atomic) component could be a function which does not call any other function

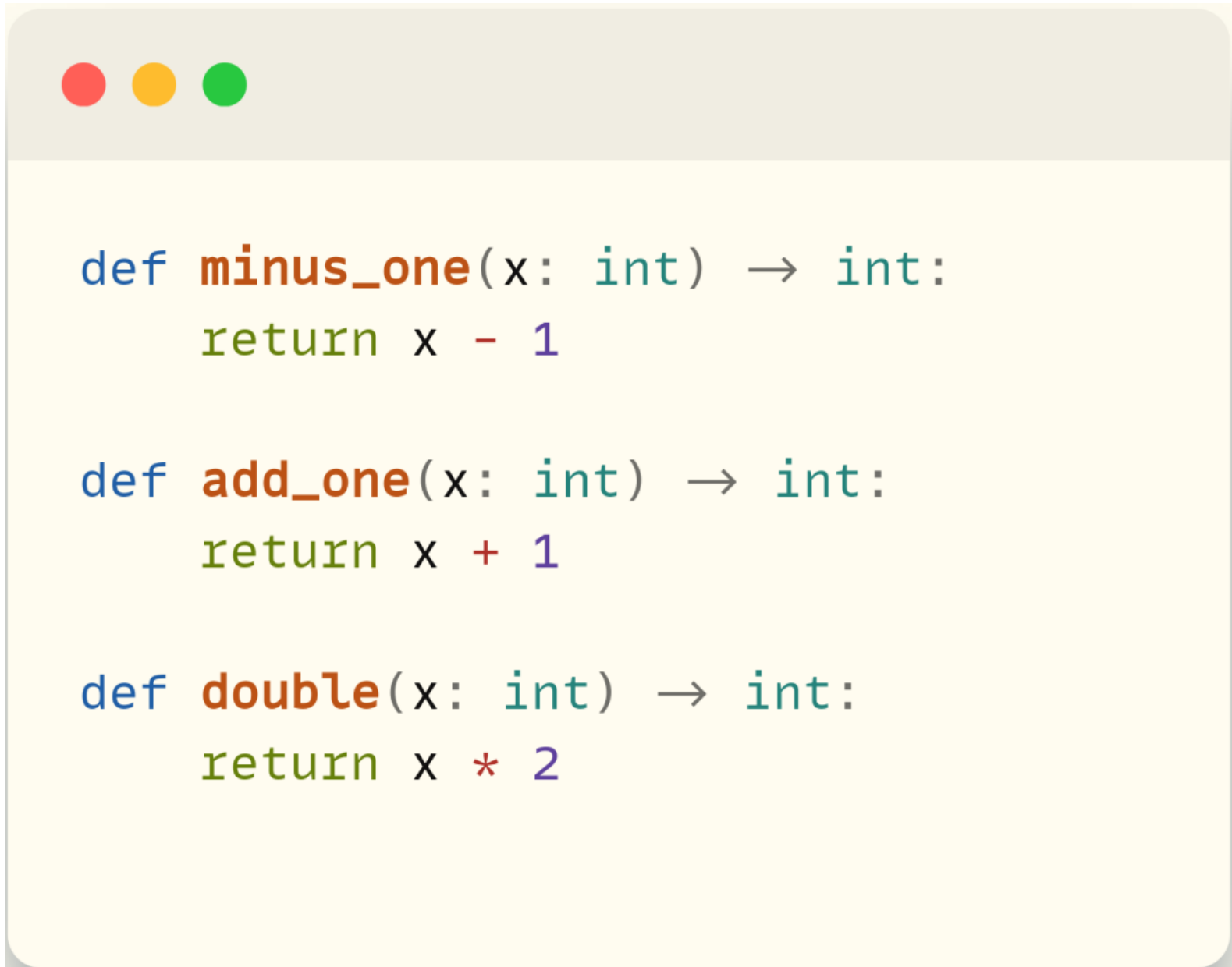
```
def minus_one(x: int) → int:  
    return x - 1
```

Interfaces: Where Things Break

- Bugs often hide in inconsistent expectations between components
- Common mismatches:
 - Units (km vs m)
 - Time steps (daily vs hourly)
 - Percentage vs decimal
 - Unexpected NaNs in data
 - Wrong data types
 - Memory leaks

What is Integration Testing?

- Example chain: $A \rightarrow B \rightarrow C$ – verify that data flows smoothly through the functions



```
def minus_one(x: int) → int:  
    return x - 1  
  
def add_one(x: int) → int:  
    return x + 1  
  
def double(x: int) → int:  
    return x * 2
```

What is Integration Testing?



```
def calculate_driver(x: int) → int:
    """
    Runs: minus_one → add_one → double
    """
    return double(add_one(minus_one(x)))
```

This is
called a
driver
function

What is Integration Testing?

```
@pytest.mark.parametrize(
    "given, expected",
    [
        (5, 10),      # ((5 - 1) + 1) × 2
        (-3, -6),     # ((-3 - 1) + 1) × 2
    ],
)
def test_calculate(given, expected):
    assert calculate_driver(given) == expected
```

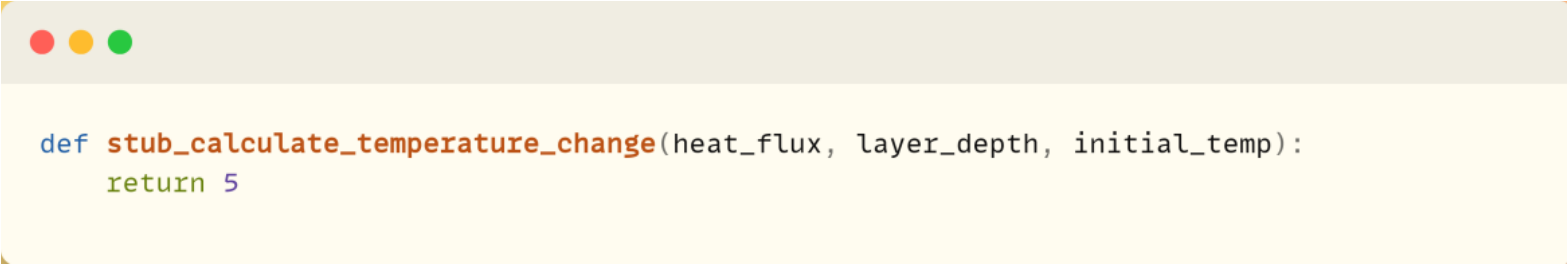

Stubbing Dependencies

- Replace real inputs (data, services, functions)
- Speeds up tests and isolates the behaviour under scrutiny

Stubbing Dependencies

```
def calculate_temperature_change(heat_flux, layer_depth, initial_temp):  
    """  
    Args:  
        heat_flux: W/m2 (positive = heat into ocean)  
        layer_depth: m  
        initial_temp: °C  
    Returns:  
        new temperature (°C)  
    """  
    mass = DENSITY_WATER * layer_depth # kg/m2  
    delta_temp = heat_flux / (mass * SPECIFIC_HEAT)  
    return initial_temp + delta_temp
```

Stubbing Dependencies

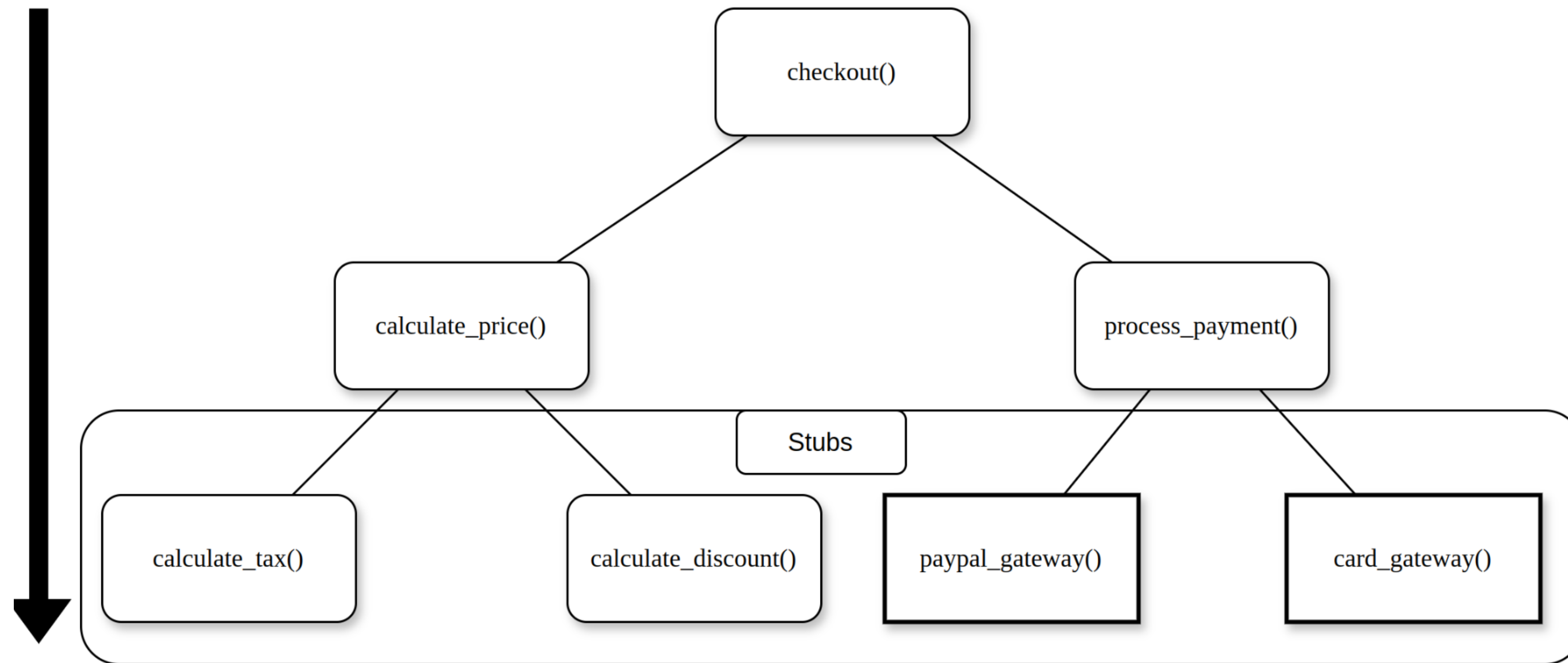


```
def stub_calculate_temperature_change(heat_flux, layer_depth, initial_temp):  
    return 5
```


Top-Down Testing

- Start with higher-level components first
- Incrementally add and test each lower layer until you reach the leaf of the tree
- Good for validating broad behaviour early

Top-Down Testing



Bottom-Up Testing

- Begin with the lowest-level components
- Build upward, testing each new layer as you integrate it
- Helpful when low-level code is more complex or you are doing a refactor

Bottom-Up Testing

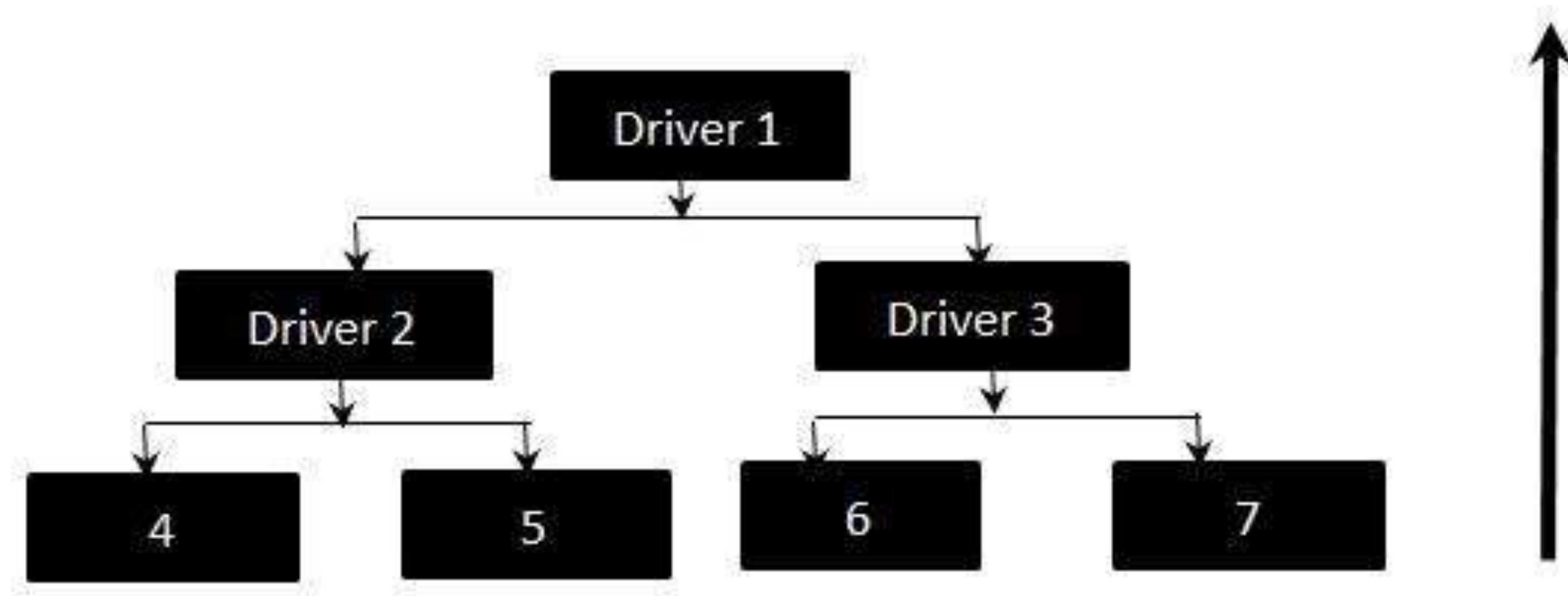


Image Credit: microsoft.github.io/code-with-engineering-playbook

Choosing an Approach

- No single style fits every project
- Pick the approach that suits your codebase, purpose, and time constraints
- Be pragmatic – mix and match when it helps

Quick Self-Checks

- What should I prioritise for integration?
 - High risk interfaces
 - Complex combinations of components
 - Components which integrate components you plan to change

End-to-End (E2E) Testing

- Run the entire system from start to finish
- Example: a complete data-analysis or climate-forecast pipeline
- Typically cover the main program flows and edge paths

End-to-End (E2E) Testing

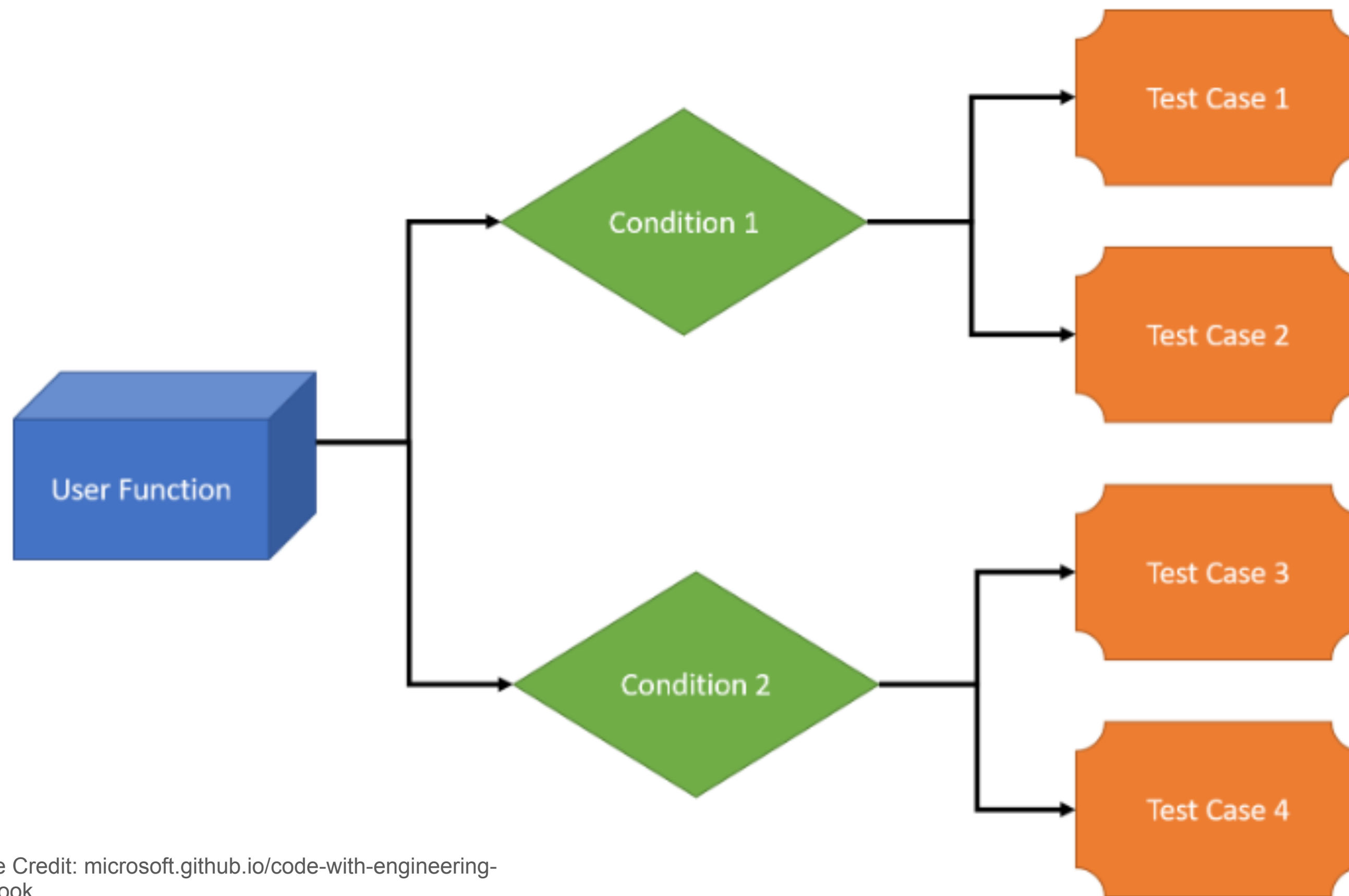


Image Credit: microsoft.github.io/code-with-engineering-playbook

End-to-End (E2E) Testing

- Pros
 - Confirms that everything works together
- Cons
 - Slowest tests
 - Hardest for localising bugs

End-to-End (E2E) Testing

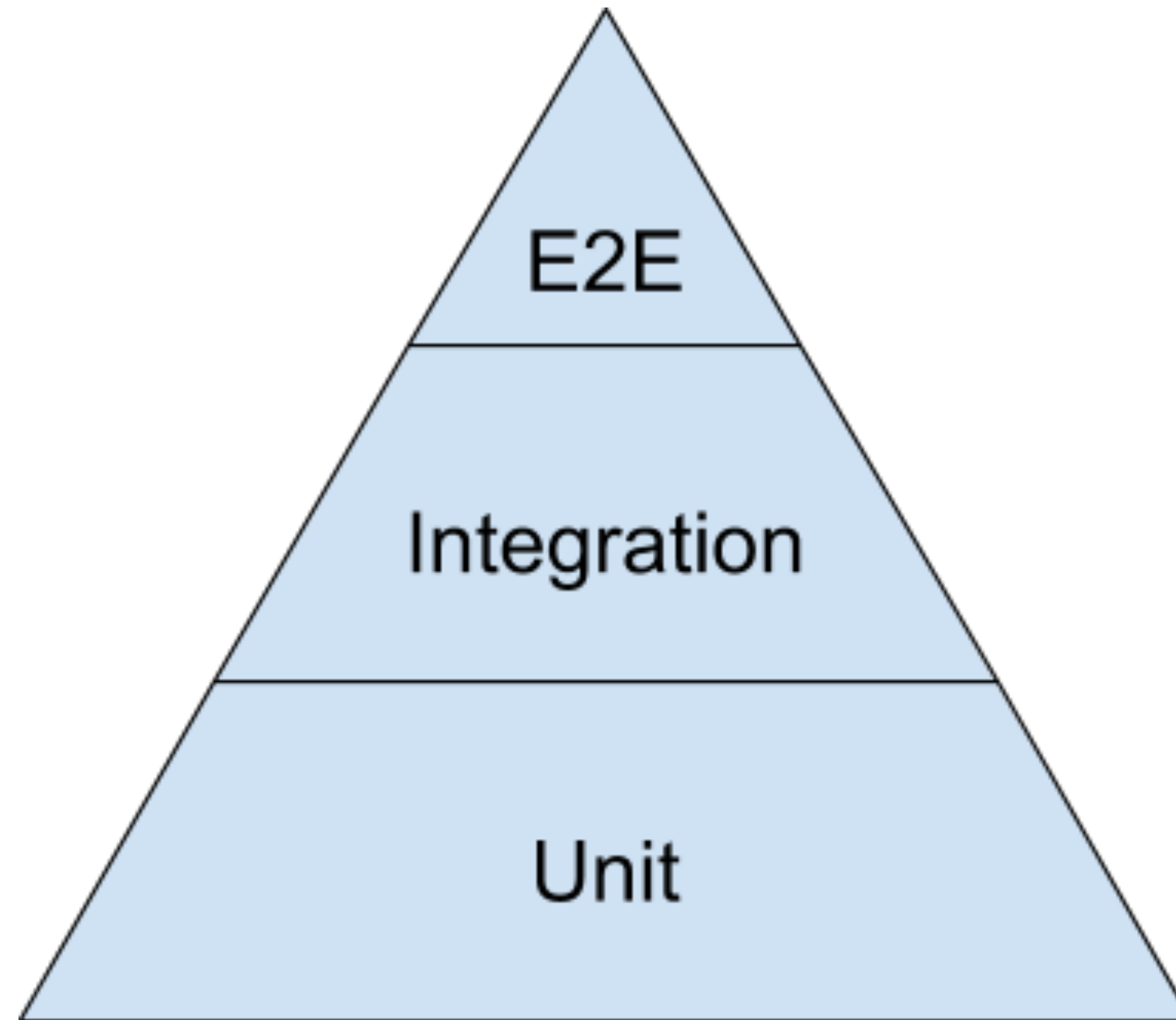


Image Credit: <https://testing.googleblog.com/2015/04/just-say-no-to-more-end-to-end-tests.html>

Summary

- Integrate early, test often, stub when appropriate
- Choose top-down, bottom-up, or a hybrid based on context
- Keep an eye on interfaces – common trip up

Part III - Property-based tests

Thanks- and happy testing!



<https://iccs.cam.ac.uk>